ALTAR OS

"The Digital Operating System for the Church"

---

1. EXECUTIVE SUMMARY

ALTAR OS is a **multi-tenant, full-stack digital infrastructure platform for churches**, enabling:

- Church management (CRM, finance, analytics)
- Member engagement (social, spiritual, communication)
- Financial systems (tithes, offerings, fundraising)
- AI-powered insights and automation
- Inter-church ecosystem and marketplace

---

2. OBJECTIVES

Primary Goals

- Digitize all church operations
- Improve engagement and retention
- Increase financial transparency and giving efficiency
- Enable scalable church growth
- Build a unified African church network

---

3. USER ROLES

3.1 Super Admin (Platform Owner)

- Manage all churches
- Monitor system health
- Approve marketplace listings
- Global analytics

3.2 Church Admin (Pastor / Leadership)

- Manage members
- Access analytics
- Handle finances
- Send communications
- Manage events

3.3 Department Leaders

- Manage groups (choir, youth, etc.)
- Track attendance
- Communicate with members

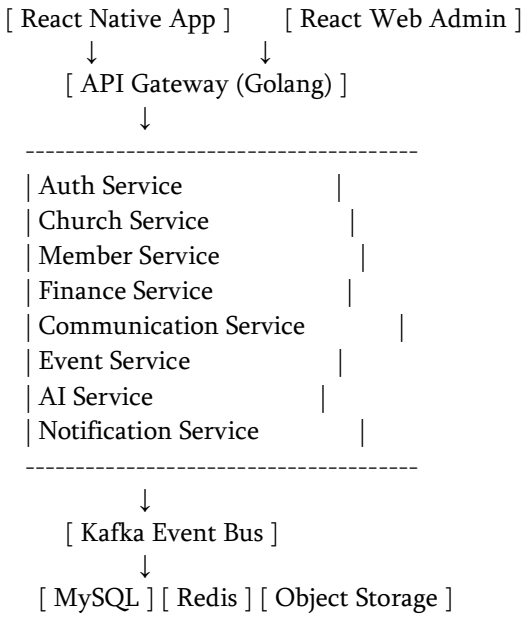
3.4 Members

- Participate in church activities
- Give offerings
- Join groups
- Access spiritual content

---

4. SYSTEM ARCHITECTURE

4.1 High-Level Architecture



4.2 Tech Stack

Frontend

- React Native (Mobile)
- React + TypeScript (Admin Dashboard)

Backend

- Golang (Microservices)
- gRPC + REST

Messaging

- Kafka (event-driven architecture)

Database

- MySQL (primary)
- Redis (caching, sessions)

Storage

- Cloudinary

Notifications

- Firebase (push)
- SMS (Africa's Talking / Hubtel)

---

5. MEMBER APP FEATURES

5.1 Authentication

- Phone number login (OTP)
- Email login
- Social login (optional)

5.2 Dashboard

- Upcoming events
- Recent sermons
- Giving summary
- Announcements

5.3 Spiritual Module

Features

- Bible (offline supported)
- Devotionals
- Sermon streaming
- Prayer requests

API

GET /devotionals
POST /prayer-request
GET /sermons

5.4 Giving System

Features

- Tithe (recurring)
- Offerings
- Donations
- Payment methods:
    - Mobile Money
    - Card
    - Crypto

API

POST /payments/initiate
POST /payments/verify
GET /payments/history

5.5 Social System

Features

- Feed (posts, testimonies)
- Comments, likes

- Group chats

API

POST /posts
GET /posts/feed
POST /comments

5.6 Events & Attendance

Features

- RSVP
- QR check-in
- Attendance logs

API

POST /events/rsvp
POST /attendance/checkin
GET /attendance/history

5.7 Welfare System

Features

- Request assistance
- Emergency alerts
- Volunteer matching

---

6. ADMIN DASHBOARD FEATURES

6.1 Church CRM

Features

- Member profiles
- Family linking
- Status tracking

Schema (MySQL)

```
CREATE TABLE members (
  id BIGINT PRIMARY KEY,
  church_id BIGINT,
  name VARCHAR(255),
  phone VARCHAR(20),
  email VARCHAR(255),
  status ENUM('active','inactive'),
  created_at TIMESTAMP
);
```

6.2 Analytics Dashboard

Metrics

- Attendance trends
- Giving trends
- Engagement score

6.3 Communication System

Features

- Broadcast messaging
- Targeted communication

Target Filters

- Location
- Department
- Activity level

6.4 Financial System

Features

- Income tracking
- Expense tracking
- Reports

Schema

```
CREATE TABLE transactions (
  id BIGINT PRIMARY KEY,
  church_id BIGINT,
  type ENUM('tithe','offering','donation'),
  amount DECIMAL(10,2),
  status VARCHAR(50),
  created_at TIMESTAMP
);
```

6.5 Project Management

Features

- Fundraising campaigns
- Donation tracking

7. AI MODULE

7.1 AI Sermon Assistant

- Input: topic
- Output: sermon outline

7.2 AI Member Insights

- Detect inactive members
- Suggest follow-ups

7.3 AI Prayer Assistant

- Chat interface
- Scripture-based responses

8. NOTIFICATION SYSTEM

Channels

- Push notifications
- SMS
- Email

Use Cases

- Event reminders
- Giving reminders
- Emergency alerts

---

9. INTER-CHURCH PLATFORM

Features

- Church discovery
- Marketplace
- Collaboration tools

---

10. SECURITY & COMPLIANCE

Security Measures

- JWT authentication
- Role-based access control (RBAC)
- Encryption (TLS)

Financial Security

- PCI-DSS compliance (via providers)
- Fraud detection

---

11. NON-FUNCTIONAL REQUIREMENTS

Performance

- Handle 1M+ users
- <200ms API response time

Scalability

- Horizontal scaling via Kubernetes

Reliability

- 99.9% uptime

---

12. MICROSERVICES BREAKDOWN

Service	Responsibility
Auth Service	Login, JWT
Church Service	Church data
Member Service	Members
Finance Service	Payments
Event Service	Events
Communication Service	Messaging
AI Service	AI features

---

13. EVENT-DRIVEN DESIGN (KAFKA)

Example Events

- member.created
- payment.completed
- attendance.checked_in
- message.sent

---

14. TESTING STRATEGY

Types

- Unit tests (Go)
- Integration tests
- Load testing

---

15. DEPLOYMENT

Infrastructure

- Docker
- Kubernetes
- CI/CD (GitHub Actions)

---

16. MONETIZATION

Revenue Streams

- SaaS subscription
- Transaction fees
- Premium features
- Marketplace commission

FINAL NOTE (VERY IMPORTANT)

This is not just a product.

If executed well:

- You control **financial flow**
- You control **communication**
- You control **community infrastructure**

That's **extremely powerful** — and sensitive.